Dictionaries

14 December 2025 22:39

The data type dictionary fall under mapping. It is a mapping between a set of keys and a set of values. The key value pair is called an item. A key is separated from its value by a colon(:) and consecutive items are separated by commas. Items in dictionaries are unordered. So we may not get back the data in the same order in which we had entered the data initially in the dictionary.

Creating a Dictionary

To create a dictionary. The items entered are separated by commas and enclosed in curly braces. Each item is a key value pair separated through colon(:). The keys in the dictionary must be unique and should be any immutable data type i.e., Int, float, string or tuple. The values can be repeated and can be of any data type.

```
>>> dict1 = {}
>>> dict1
{}
>>> dict2 = {}
>>> dict2
{}
>>> dict3 = {'Mohan':95, 'Jansen':89, 'Adrean':78, 'Ellis':71, 'Karan':91}
>>> dict3
{'Mohan': 95, 'Jansen': 89, 'Adrean': 78, 'Ellis': 71, 'Karan': 91}
```

Key : value

Key

value.

Accessing Items in a Dictionary

As we have already. Seen that the items of a sequence. (String, list and tuple) Are accessed using a technique called indexing. The items of a dictionary are accessed via the keys rather than via their relative positions or indices. Each key serves as the index and maps to a value. The following example shows how a dictionary returns the value corresponding to the given key:

```
>>> dict3
{'Mohan': 95, 'Jansen': 89, 'Adrean': 78, 'Ellis': 71, 'Karan': 91}
>>> dict3['Adrean']
78
>>> dict3['Karan']
91
```

```
>>> dict3['Liam']
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    import platform
    ^^^^^^^^^^^^^^
KeyError: 'Liam'
```

Key not present in the dictionary.

Dictionaries are Mutable

Dictionaries are mutable, which implies that the content of the dictionary can be changed after it has been created.

Adding a new item and modifying the existing item

We can add a new item to the dictionary as shown in the following example:

```
>>> dict3
{'Mohan': 95, 'Jansen': 89, 'Adrean': 78, 'Ellis': 71, 'Karan': 91}
>>> dict3['Ellis'] = 80
>>> dict3
{'Mohan': 95, 'Jansen': 89, 'Adrean': 78, 'Ellis': 80, 'Karan': 91}
>>> dict3['Amy'] = 77
>>> dict3
{'Mohan': 95, 'Jansen': 89, 'Adrean': 78, 'Ellis': 80, 'Karan': 91, 'Amy': 77}
```

Modifying existing item.

adding existing item.

```
>>> dict3
{'Mohan': 95, 'Jansen': 89, 'Adrean': 78, 'Ellis': 71, 'Karan': 91}
>>> dict3['Ellis'] = 80 <-- Modifying existing item.
>>> dict3
{'Mohan': 95, 'Jansen': 89, 'Adrean': 78, 'Ellis': 80, 'Karan': 91}
>>> dict3['Amy'] = 77 <-- adding existing item.
>>> dict3
{'Mohan': 95, 'Jansen': 89, 'Adrean': 78, 'Ellis': 80, 'Karan': 91, 'Amy': 77}
>>> dict3['Vaanya'] = 90
>>> dict3
{'Mohan': 95, 'Jansen': 89, 'Adrean': 78, 'Ellis': 80, 'Karan': 91, 'Amy': 77, 'Vaanya': 90}
```

Dictionary Operations

Membership: The membership operator **in** checks if the key is present in the dictionary and returns True, else it returns False.

The **not in** operator returns **True** if the key is not present in the dictionary, else it returns **False.**

```
>>> dict3
{'Mohan': 95, 'Jansen': 89, 'Adrean': 78, 'Ellis': 80, 'Karan': 91, 'Amy': 77, 'Vaanya': 90}
>>> 'Sushant' in dict3
False
>>> 'Vaanya' in dict3
True
>>> 'Sushant' not in dict3
True
>>> 'Vaanya' not in dict3
False
```

Traversing a Dictionary

So we can access each item of the dictionary or traverse a dictionary using for loop.

Method-1

```
>>> for key in dict3:
...     print(key,':',dict3[key],sep='')
...
Mohan:95
Jansen:89
Adrean:78
Ellis:80
Karan:91
Amy:77
Vaanya:90
```

Method-2

```
>>> for key,value in dict3.items(): <-- This methods returns key, value pair
...     print(key,':',value,sep='')
...
Mohan:95
Jansen:89
Adrean:78
Ellis:80
Karan:91
Amy:77
Vaanya:90
```

Dictionary methods and built in functions.

Python provides many functions to work on dictionaries:

Method	Description	Example
len()	Returns the length or	>>> dict3

	number of key:value pair of the dictionary passed as the argument.	{'Mohan': 95, 'Jansen': 89, 'Adrean': 78, 'Ellis': 80, 'Karan': 91, 'Amy': 77, 'Vaanya': 90} >>> len(dict3) 7

dict()	Creates a dictionary from a sequence of key-value pairs.	>>> pairs = [('Sammy',[20, 30, 21]), ('Frank',[21,25,30])]
		>>> pairs
		[('Sammy', [20, 30, 21]), ('Frank', [21, 25, 30])]
		>>> dict1 = dict(pairs)
		>>> dict1
		{'Sammy': [20, 30, 21], 'Frank': [21, 25, 30]}
		>>> dict1['Sammy']
		[20, 30, 21]

keys()	The returns a list of keys in the dictionary.	>>> dict3
		{'Mohan': 95, 'Jansen': 89, 'Adrean': 78, 'Ellis': 80, 'Karan': 91, 'Amy': 77, 'Vaanya': 90}
		>>> k = dict3.keys()
		>>> k
		dict_keys(['Mohan', 'Jansen', 'Adrean', 'Ellis', 'Karan', 'Amy', 'Vaanya'])

values()	He returns a list of values in the dictionary.	>>> dict3
		{'Mohan': 95, 'Jansen': 89, 'Adrean': 78, 'Ellis': 80, 'Karan': 91, 'Amy': 77, 'Vaanya': 90}
		>>> v = dict3.values()
		>>> v
		dict_values([95, 89, 78, 80, 91, 77, 90])

items()	Returns a list of tuple (key-value) pair.	>>> t = dict3.items()
		>>> t
		dict_items([('Mohan', 95), ('Jansen', 89), ('Adrean', 78), ('Ellis', 80), ('Karan', 91), ('Amy', 77), ('Vaanya', 90)])

get()	Returns the value corresponding to the key passed as the argument. If the key is not present in the dictionary, it will return None.	>>> dict3
		{'Mohan': 95, 'Jansen': 89, 'Adrean': 78, 'Ellis': 80, 'Karan': 91, 'Amy': 77, 'Vaanya': 90}
		>>> dict3.get('Ellis')
		80
		>>> v = dict3.get('Sushant')
		>>> v
		>>> print(v)
		None

update()	Appends the key:value pair of the dictionary passed as the argument to the key:value pair of the given dictionary.	>>> dict1
		{'Sammy': [20, 30, 21], 'Frank': [21, 25, 30]}
		>>> dict3
		{'Mohan': 95, 'Jansen': 89, 'Adrean': 78, 'Ellis': 80, 'Karan': 91, 'Amy': 77, 'Vaanya': 90}
		>>> dict1.update(dict3)
		>>> dict1
		{'Sammy': [20, 30, 21], 'Frank': [21, 25, 30], 'Mohan': 95, 'Jansen': 89, 'Adrean': 78, 'Ellis': 80, 'Karan': 91, 'Amy': 77, 'Vaanya': 90}
		>>> dict3
		{'Mohan': 95, 'Jansen': 89, 'Adrean': 78, 'Ellis': 80, 'Karan': 91, 'Amy': 77, 'Vaanya': 90}

del()	Deletes the item with the given key.	>>> dict1 {'Sammy': [20, 30, 21], 'Frank': [21, 25, 30],

	To delete the dictionary from the memory we write: del Dict_name.	'Mohan': 95, 'Jansen': 89, 'Adrean': 78, 'Ellis': 80, 'Karan': 91, 'Amy': 77, 'Vaanya': 90}
		>>> del dict1['Frank']
		>>> dict1
		{'Sammy': [20, 30, 21], 'Mohan': 95, 'Jansen': 89, 'Adrean': 78, 'Ellis': 80, 'Karan': 91, 'Amy': 77, 'Vaanya': 90}
		>>> del dict1 ← it removes the object from the memory.
		>>> dict1
		Traceback (most recent call last):
		File "<stdin>", line 1, in <module>
		import platform
		^^^^^ ← deleted from memory
		NameError: name 'dict1' is not defined. Did you mean: 'dict2'?

clear()	Deletes or clears all the items of the dictionary.	>>> dict1
		{'Sammy': [20, 30, 21], 'Frank': [21, 25, 30], 'Mohan': 95, 'Jansen': 89, 'Adrean': 78, 'Ellis': 80, 'Karan': 91, 'Amy': 77, 'Vaanya': 90}
		>>> dict1.clear()
		>>> dict1 ← object is not removed from the memory.
		{}

Manipulating Dictionaries

Create a dictionary odd of odd and numbers between one and 10. Where the key is the decimal number and the value is the corresponding number in words. Perform the following operations on this dictionary.

i. Display the keys.
ii. Display the items.
iii. Find the length of the dictionary.
iv. Check if 7 is present or not.
v. Check if two is present or not.
vi. Retrieve the value corresponding to the key 9.
vii. Delete the item from the dictionary corresponding to the key 9.

```
# Write a program to enter names of employees and their salaries as input and store
# them in a dictionary
num = int(input('Enter number of employees whose data to be stored: '))
count = 1
employee = dict()
while count <= num:
    name = input('Enter the name of employee: ')
    salary = int(input('Enter the salary: '))
    employee[name] = salary
    count += 1
print('\n\nEmployee Name \t Salary')
for k,v in employee.items():
    print(k,'\t\t', v)
```

Output:

```
Enter the name of employee: Vaanya
Enter the salary: 30000
Enter the name of employee: Sushant
Enter the salary: 32000
Enter the name of employee: Dean
Enter the salary: 29000


Employee Name    Salary
Vaanya           30000
Sushant                 32000
Dean             29000
```

Homework

1. Student Marks Management System

Write a Python program to create a dictionary that stores the names of students as keys and their marks in three subjects as values (stored as a list).

The program should:

- **Accept details for *n* students from the user.**
- **Display all student names.**
- **Display the complete dictionary.**
- **Calculate and display the average marks of each student.**
- **Find and display the student who scored the highest total marks.**
- **Check whether a given student name exists in the dictionary or not.**
- **Delete a student's record based on user input.**

2. Employee Salary Analyzer

Create a dictionary to store employee names as keys and their monthly salaries as values.

The program should:

- **Accept employee details from the user.**
- **Display all employees earning more than a user-specified salary.**
- **Find and display the highest and lowest salary.**
- **Increase the salary of all employees by 10%.**
- **Allow the user to update the salary of a specific employee.**
- **Display the final dictionary after all modifications.**

3. Word Frequency Counter

Write a program that takes a paragraph of text as input and stores each word as a key and its frequency as the value in a dictionary.

The program should:

- **Ignore case sensitivity.**
- **Display all words and their frequencies.**
- **Display only those words that occur more than once.**
- **Find the most frequently occurring word.**
- **Remove a word entered by the user from the dictionary (if it exists).**

4. Odd Number Dictionary Manipulation

Create a dictionary containing odd numbers between 1 and 20 as keys and their corresponding number names as values (e.g., 3: "three").

Perform the following operations:

- **Display all keys.**
- **Display all values.**
- **Display all key-value pairs.**

- Check whether key 15 exists.
- Retrieve the value corresponding to key 9.
- Delete the item with key 9.
- Display the length of the dictionary after deletion.

5. Library Book Inventory
Create a dictionary where book IDs are keys and book titles are values.
The program should:
- Add *n* books to the dictionary.
- Display all available books.
- Search for a book using its ID.
- Update the title of a book.
- Delete a book using its ID.
- Clear the entire inventory on user confirmation.

6. Student Result Report
Create a dictionary where roll numbers are keys and marks are values.
The program should:
- Accept student data from the user.
- Display all roll numbers and marks.
- Calculate and display the class average.
- Identify students who scored below the average.
- Check whether a particular roll number exists.
- Use the get() method to safely retrieve marks for a given roll number.

7. Merge Two Dictionaries
Write a program to create two dictionaries:
- One containing student names and marks.
- Another containing student names and grades.
The program should:
- Merge both dictionaries using the update() method.
- Display the merged dictionary.
- Count the total number of items.
- Display only the keys and only the values separately.
- Delete one specific student record from the merged dictionary.

8. Product Price Management
Create a dictionary with product names as keys and prices as values.
The program should:
- Accept product details from the user.
- Display products priced above a given value.
- Apply a discount of 5% to all products.
- Allow the user to remove a discontinued product.
- Display the updated price list.

9. Dictionary Traversal and Analysis
Create a dictionary storing city names as keys and their population (in lakhs) as values.
The program should:
- Traverse the dictionary using two different methods.
- Display cities with population greater than a given value.
- Count the total number of cities.
- Check if a specific city exists using membership operators.
- Clear all entries from the dictionary at the end.

10. Menu-Driven Dictionary Program

Write a menu-driven Python program to perform the following operations on a dictionary:

1. Add a new key–value pair
2. Display all items
3. Search for a key
4. Update an existing value
5. Delete a key
6. Display keys, values, and items separately
7. Exit

Ensure proper use of dictionary methods like keys(), values(), items(), get(), del, and len().